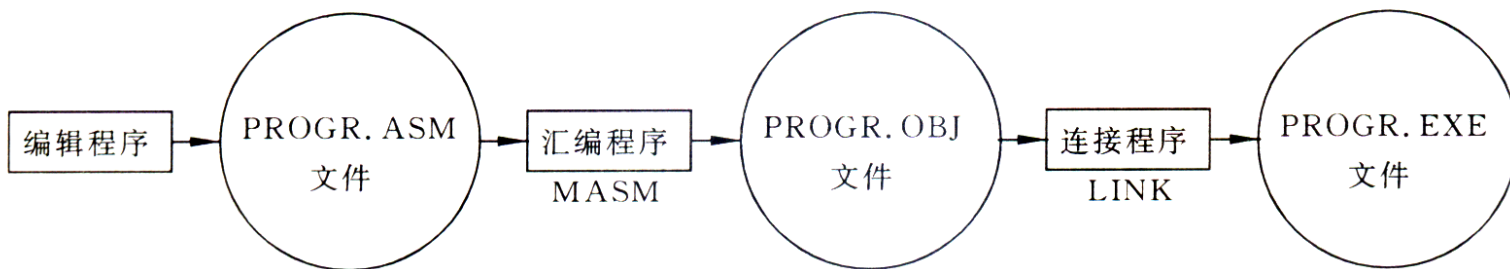




第4章 汇编语言及其程序设计

4.1 汇编语言概述

- **指令**：指使计算机完成某种操作的命令。
- **程序**：完成某种功能的指令序列。
- **机器语言**：计算机能直接识别的语言。用机器语言写出的程序称为机器代码。
- **汇编语言**：用字符记号代替机器指令。用汇编语言编写的程序叫汇编语言源程序。
- **汇编程序**：一种翻译程序，把助记符翻译成机器语言。
- 在计算机上运行汇编语言程序的步骤：





4.2 汇编语言语句格式

- 汇编语句格式有4个字段：

[名字] 操作符 操作数； [注释]

1. 一种标识符。

2. **组成：**

A~Z, a~z;

0~9;

专用符号 ? . @ _ \$

3. **限制：**

1) 第一个字符不能是数字;

2) “.” 必须是第一个字符;

3) 前31个字符有效;

4) 不能为关键字。

4. **类型：** 标号-指令符号地址
变量-数据符号地址

说明程序、指令的功能，
增加程序的可读性

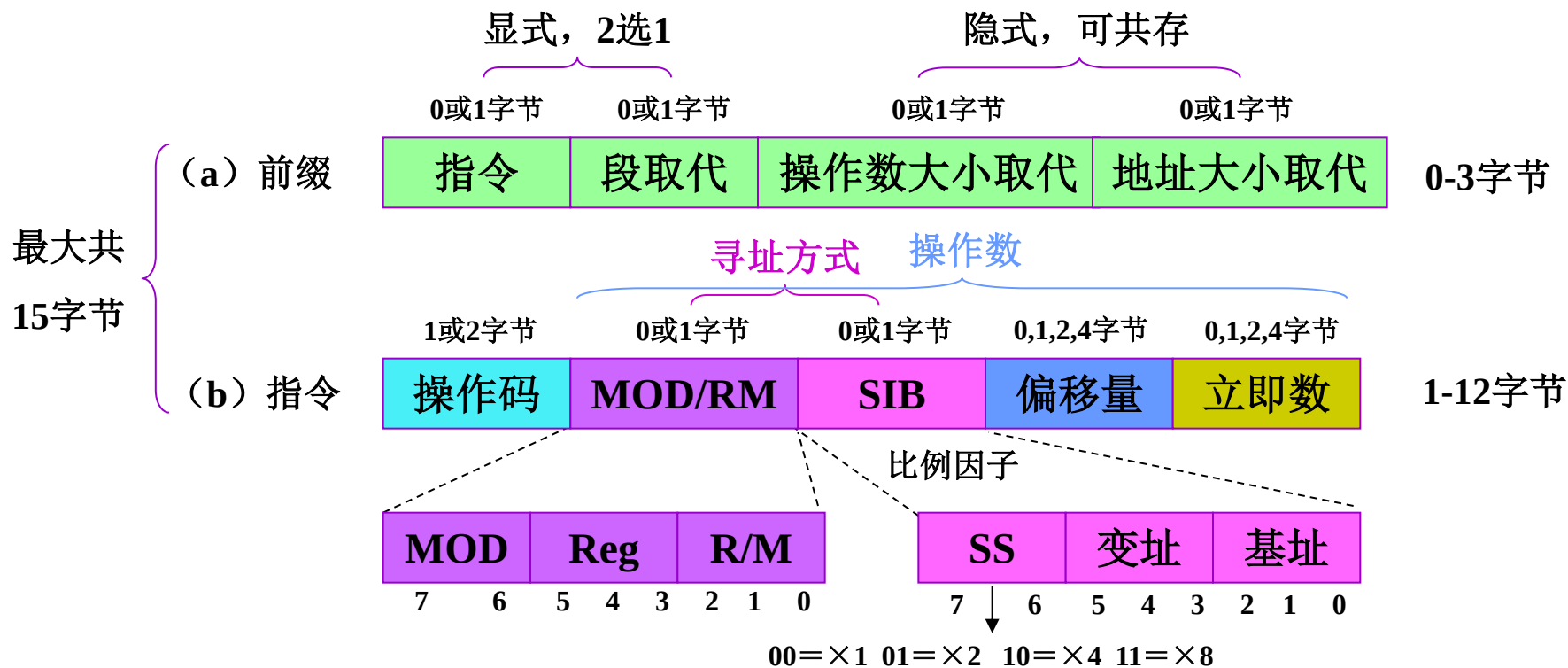
指定参与操作的数据，
或数据所在单元地址

组成： CPU指令、伪指令、宏指令



4.3 指令格式

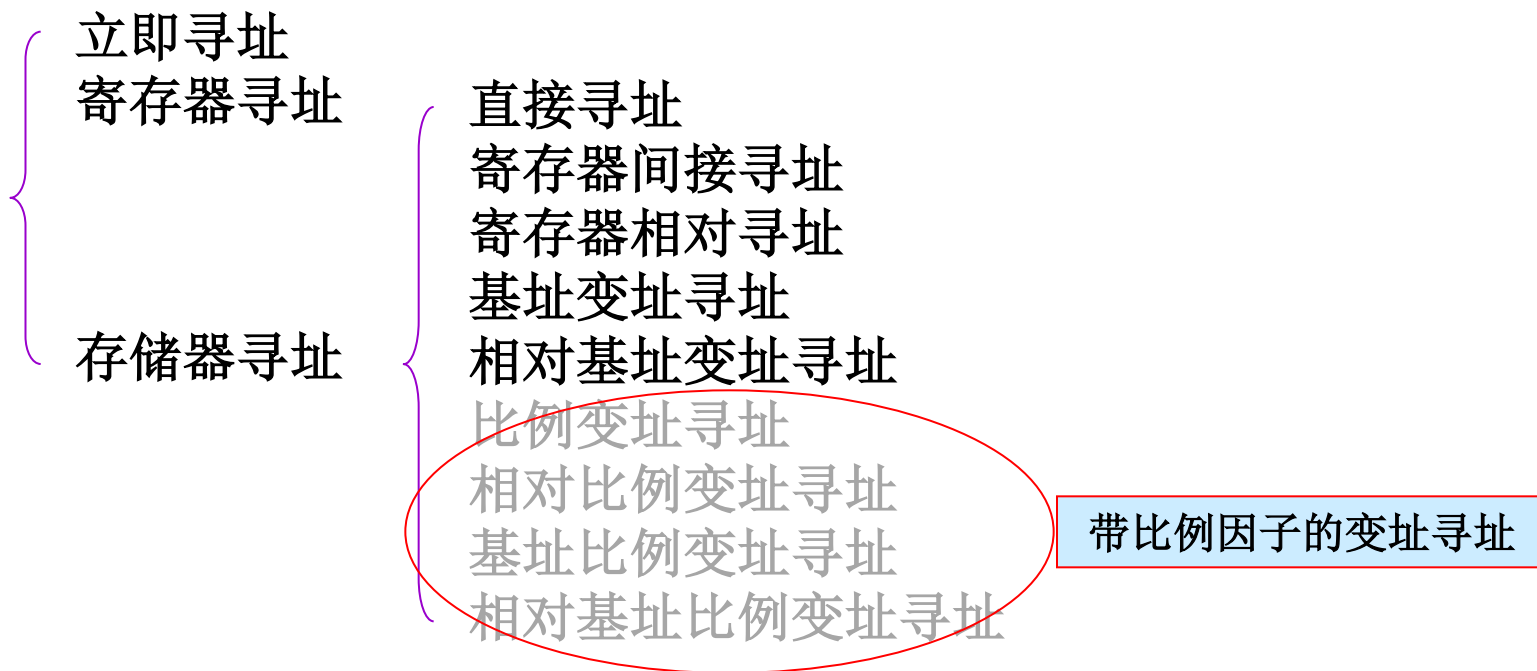
- 一条指令由五部分组成：前缀、操作码、**寻址方式**、**偏移量**、**立即数**。





4.4 寻址方式

- 有2类寻址：
 - (1) 对操作数寻址
 - (2) 对转移地址、调用地址寻址
- Pentium系统中使用字节（8位）、字（16位）、双字（32位）和4倍字（64位）。采用低端存储方式。
- 与数据有关的寻址方式有11种：





1. 立即寻址

- 立即寻址：
 - 操作数是指令的一部分。完整地取出该条指令之后也就获得了操作数。
- 汇编语言规定：
 - 立即数必须以数字开头；
 - 以字母开头的十六进制数前必须以数字0做前缀；
 - 数制用后缀表示，B表示二进制数，H表示十六进制数，D或者缺省表示十进制数，Q表示八进制数。
- 汇编源程序在汇编时对立即数的处理：
 - 对于不同进制的立即数汇编成等值的二进制数，有符号数以补码表示。
- 例：
MOV AL, 01010101B ; 二进制立即数01010101 B送寄存器AL
MOV BX, 0AB12H ; 十六进制立即数0AB12H (16位二进制数) 送寄存器BX



2. 寄存器寻址

- 寄存器寻址：

- 操作数在CPU的某个寄存器中，指令指定寄存器号。
- 对于8位操作数，寄存器有AH、AL、BH、BL、CH、CL、DH和DL；
- 对于16位操作数，寄存器可以是AX、BX、CX、DX、SP、BP、SI和DI；
- 对于32位操作数，寄存器有EAX、EBX、ECX、EDX、ESP、EBP、EDI和ESI。

- 对段寄存器的寻址：

- MOV、PUSH和POP指令可寻址16位的段寄存器(CS、ES、DS、SS、FS和GS)。

- 例：

MOV DS, AX ; AX寄存器内容送段寄存器 DS

MOV DL, BH ; 8位寄存器BH内容送8位寄存器DL

MOV AX, 12 ; 十进制数12（转换为16位二进制数）送寄存器AX



3. 存储器寻址

- **存储器寻址**：操作数在存储器中。如何取得操作数存储位置逻辑地址的偏移地址，在寻址中称为有效地址EA。
- **有效地址EA**：Effective Address，在8086～Pentium微处理器里，把操作数的**偏移地址**称为有效地址EA。
- **对于8086**： $EA = \begin{bmatrix} BX \\ BP \end{bmatrix} + \begin{bmatrix} SI \\ DI \end{bmatrix} + \begin{bmatrix} 8\text{位位移量} \\ 16\text{位位移量} \end{bmatrix}$ ；（每个部分最多保留一项，至少有一项有值）
- **寻址方式的段约定和段超越**：
 - 使用BP、SP（EBP、ESP）寻址，默认的段为SS；其他通用寄存器参与寻址，默认的段为DS；取指默认的段为CS；数据串操作目的地址寻址默认的段为ES，源地址寻址默认的段为DS。



有效地址的计算方法

- 有效地址的计算公式：

$$EA = \text{基址} + (\text{变址} \times \text{比例因子}) + \text{位移量}$$

- 基址：**基址寄存器中的内容。基址寄存器通常用于编译程序指向局部变量区、数据段中数组或字符串的**首地址**。
- 变址：**变址寄存器中的内容。变址寄存器用于**访问**数组或字符串的**元素**。
- 比例因子：**32位机的寻址方式。寻址中，用变址寄存器的内容乘以比例因子得到变址值。该方式对访问数组特别有用。
- 位移量：**指令中的一个数，但不是立即数，而是一个地址。
- 8086/286**是16位寻址，**80386**及以后机型是32位寻址，也可用16位寻址。

四种成分	16位寻址	32位寻址
位移量	0、8、16位	0、8、32位
基址寄存器	BX、BP	任何32位通用寄存器
变址寄存器	SI、DI	除ESP以外的任何32位通用寄存器
比例因子	无	1、2、4、8



表 4.4.1 存储器寻址方式

寻址方式	基址	变址	比例因子	位移量
1. 直接寻址				✓
2. 寄存器间接寻址	✓*	✓*		
3. 寄存器相对寻址	✓*	✓*		✓
4. 基址变址寻址	✓	✓		
5. 相对基址变址寻址	✓	✓		✓
6. 比例变址寻址		✓	✓	
7. 相对比例变址寻址		✓	✓	✓
8. 基址比例变址寻址	✓	✓	✓	
9. 相对基址比例变址寻址	✓	✓	✓	✓

一对，
寻址及
相对寻址

四种成分	16位寻址	32位寻址
位移量	0、8、16位	0、8、32位
基址寄存器	BX、BP	任何32位通用寄存器
变址寄存器	SI、DI	除ESP以外的任何32位通用寄存器
比例因子	无	1、2、4、8

任取一个



4. 转移地址的寻址方式

- **转移地址：**转移指令中的操作数是转移的目的地址，称为转移地址。
- **转移地址寻址：**用于确定转移指令、CALL指令的转向地址。
- **转移/调用指令类型：**

转移指令 调用指令	{	段内 (IP/EIP)	{	相对
				间接
	{	段间 (CS:IP/EIP)	{	直接
				间接

段内转移，只修改指令指针寄存器IP/EIP；

段间转移，修改IP/EIP及代码段寄存器CS。

相对寻址：给出修改指令指针的一个有符号数（位移量）；

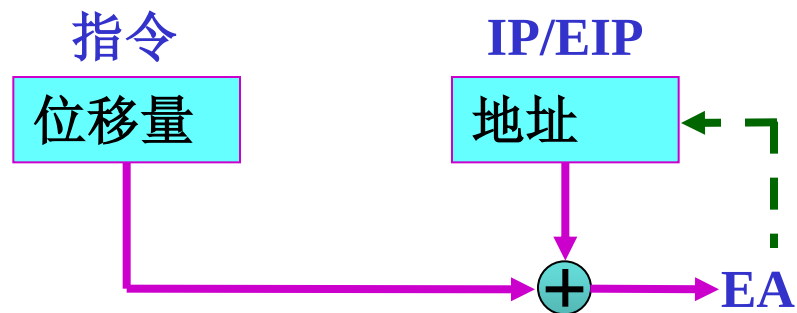
直接寻址：目标地址直接给出；

间接寻址：地址在寄存器或存储器中。



1) 段内相对寻址

- 位移量+IP/EIP =EA → IP/EIP
 - 位移量是相对值，所以称为段内相对寻址。
 - 用于条件转移、无条件转移，条件转移只能用此方式。



短跳转例：

JMP SHORT A1 ; 转到标号A1对应地址执行，A1与本指令地址差用 8位有符号表示。

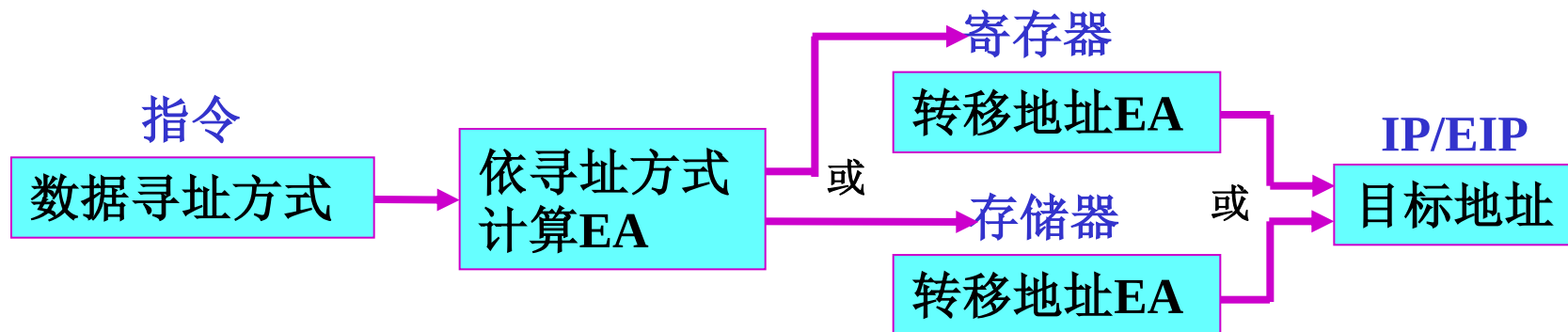
近跳转例：

JMP NEAR PTR A2 ; 转到标号A2对应地址执行，A1与本指令地址差用 16/32位有符号表示。



2) 段内间接寻址

- 寄存器/存储器的内容 = $EA \rightarrow IP/EIP$
- 这种寻址方式不能用于条件转移指令。



- 例:

JMP BX ; 寄存器BX内容送IP。

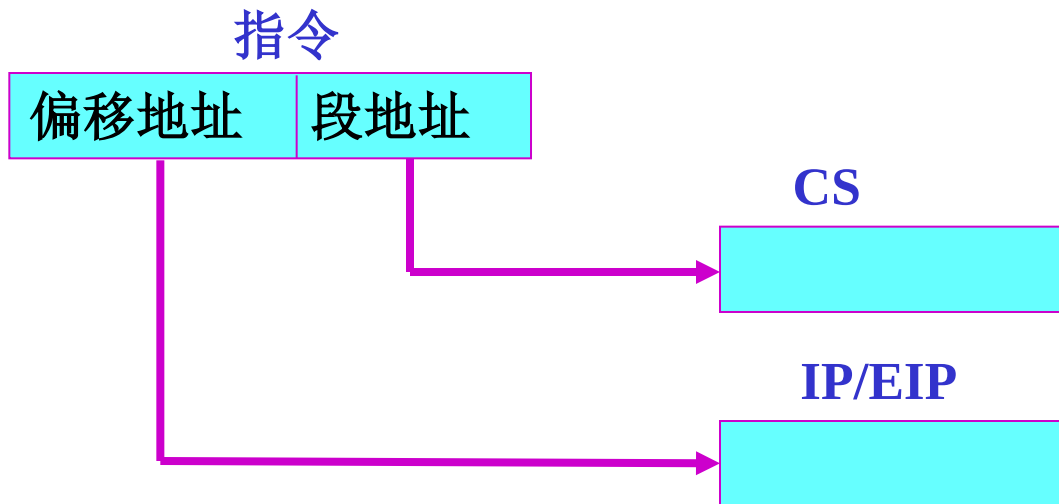
JMP WORD PTR [BX+2] ; 存储器单元[BX+2]开始存放的一个字数据送给IP。



3) 段间直接寻址

- 目标段地址:偏移量 → CS:IP/EIP

指令中直接给出目的
地址的逻辑地址



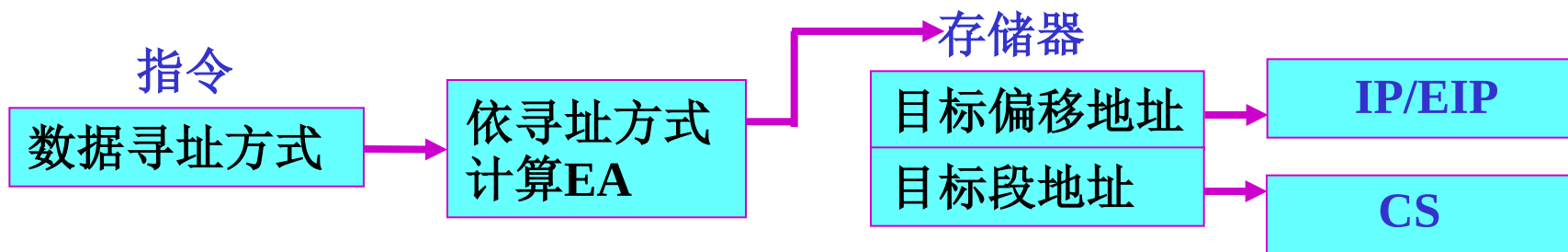
- 例:

JMP FAR PTR AA6 ; 标号AA6对应的段地址送CS,
段内偏移送IP/EIP。



4) 段间间接寻址

- 连续的存储器单元内容 → CS:IP/EIP



DWORD~段间, WORD~段内

- 例:

JMP **DWORD PTR** [SI] ; [SI]指向的字送入IP,
[SI+2] 指向的字送入CS。



5. 堆栈地址寻址

- **堆栈：**以“先进后出”方式工作的一个特定的存储区。
 - 一端固定，称为栈底；另一端浮动，称为栈顶，只有一个出入口。
- **堆栈作用：**保存传递参数、现场参数、寄存器内容、返回地址。
- **80x86规定：**
 - (1) 堆栈向小地址方向增长。
 - (2) 必须使用堆栈段，由堆栈段寄存器SS给出段基址。
 - (3) **PUSH指令压入数据时，先修改指针，再按照指针指示的单元存入数据。**
 - (4) **POP指令弹出数据时，先按照指针指示的单元取出数据，再修改指针。**
 - (5) 压入和弹出的数据类型（数据长度）不同，堆栈指针修改的数值（数据的字节数）也不同。



4.5 指令系统

- **指令系统：**指令的集合。软硬件交界面。
- **指令系统掌握要点：**指令的功能、使用了哪些寄存器或存储器单元、改变了哪些寄存器或存储器单元、对标志位的影响、基本应用方法。侧重掌握指令的功能和使用方法，为后续程序设计打基础。
- **指令按功能分为10类：**
 - 数据传送指令、算术运算指令、BCD 码调整指令、逻辑运算指令、**
 - 移位循环指令、控制转移指令、**条件设置指令、串操作指令、处理器控制指令、保护模式系统控制指令。



4.5.1 数据传送指令

- **2. PUSH 进栈** (push onto stack)
- **格式:** PUSH SRC ; 堆栈 $\leftarrow$ (SRC)
- **操作:**

16位指令: $(SP) \leftarrow (SP) - 2$
 $((SP)+1, (SP)) \leftarrow (SRC)$

32位指令: $(ESP) \leftarrow (ESP) - 4, ((ESP)+3, (ESP)+2, (ESP)+1, (ESP)) \leftarrow (SRC)$

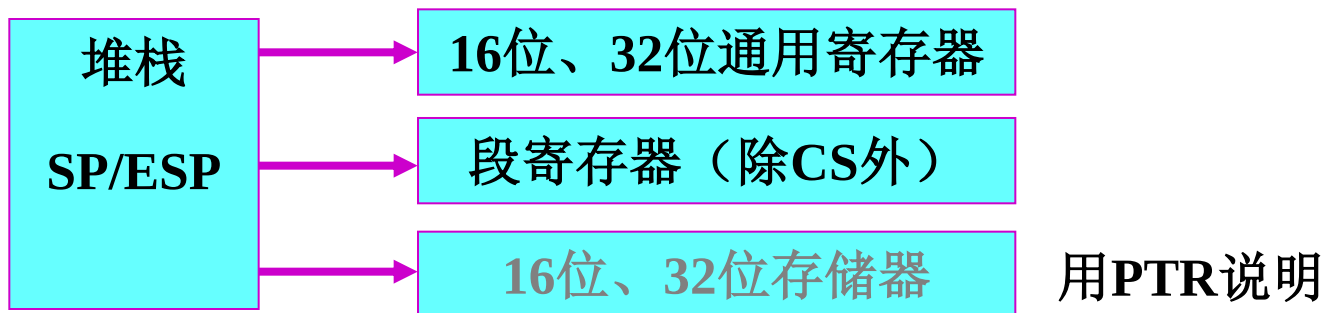
类型:





4.5.1 数据传送指令

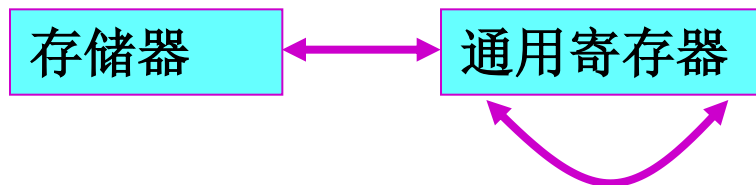
- **3. POP 出栈** (pop from the stack)
- **格式:** POP DST ; (DST) $\leftarrow$ 堆栈栈顶内容
- **操作:**
 - 16位指令: (DST) $\leftarrow$ ((SP)+1, (SP))
(SP) $\leftarrow$ (SP)+2
 - 32位指令: (DST) $\leftarrow$ ((ESP)+3, (ESP)+2, (ESP)+1, (ESP)),
(ESP) $\leftarrow$ (ESP)+4
- **类型:**





4.5.1 数据传送指令

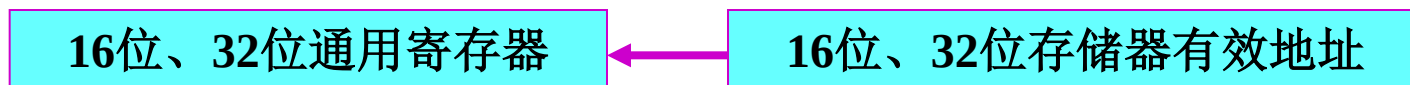
- **4. PUSHF/POPF 16位标志寄存器进栈/出栈**
- 格式: **PUSHF**
- 功能: 标志寄存器低16位压入堆栈
- 格式: **POPF**
- 功能: 从栈顶弹出2个字节送标志寄存器低16位
- **5. XCHG 交换**
- 格式: **XCHG OPR1, OPR2**
- 功能: $(\text{OPR1}) \longleftrightarrow (\text{OPR2})$
- 类型:





4.5.1 数据传送指令

- **6. LEA 有效地址送寄存器** (load effective address)
- **格式:** LEA REG, SRC ; (REG) $\leftarrow$ (SRC)的有效地址
- **类型:**



- **例:**

1 MOV AX, MEM ;AX=3412H
2 LEA AX, MEM ;AX=3100H

MEM 2000:3100	12H
3101	34H
3102	56H
3103	78H
3104	

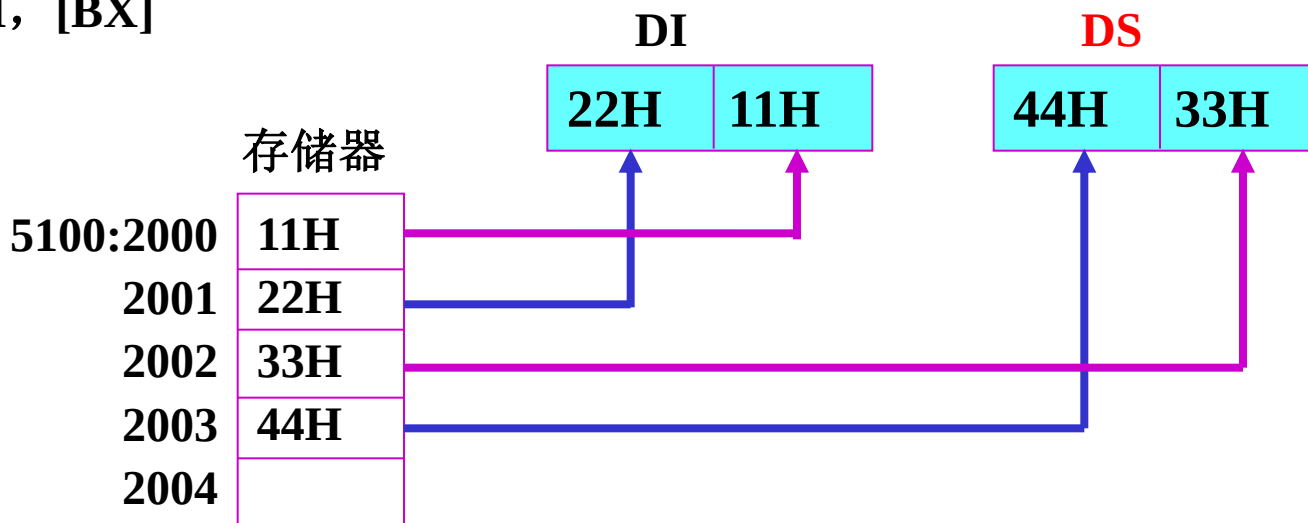


4.5.1 数据传送指令

7. LDS、LES、LFS、LGS和LSS（指针送寄存器和段寄存器）

- 格式: **LDS** REG, SRC
- 功能: $(\text{REG}) \leftarrow [\text{SRC}]$
 $(\text{DS}) \leftarrow [\text{SRC}+2]$ (16位EA)
 $(\text{DS}) \leftarrow [\text{SRC}+4]$ (32位EA)
- 例: 已知 $(\text{DS})=5100\text{H}$, $(\text{BX})=2000\text{H}$

LDS DI, [BX]





4.5.1 数据传送指令

8. XLAT 换码

- 格式: XLAT
- 功能: 完成代码转换, 偏移量8位, 表长 ≤ 256

16位指令 $(AL) \leftarrow [(BX) + (AL)]$

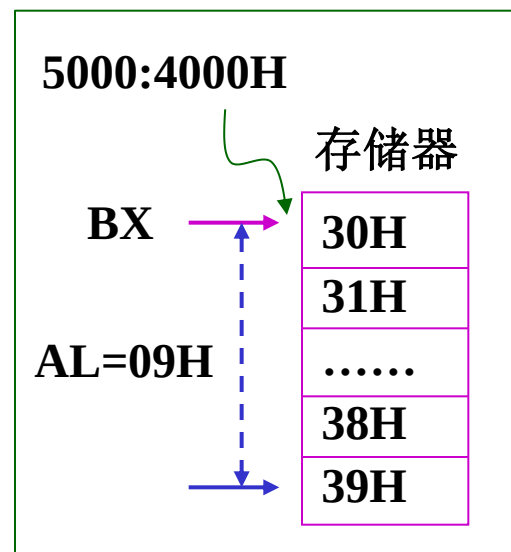
32位指令 $(AL) \leftarrow [(EBX) + (AL)]$

- 例:

非压缩BCD码转换成ASCII码, $09H \rightarrow 39H$ 。

已知 $(DS)=5000H$, $(BX)=4000H$, $(AL)=09H$, $5000:4000H$ 开始存储数字ASCII转换表。

执行 XLAT ; $AL=39H$





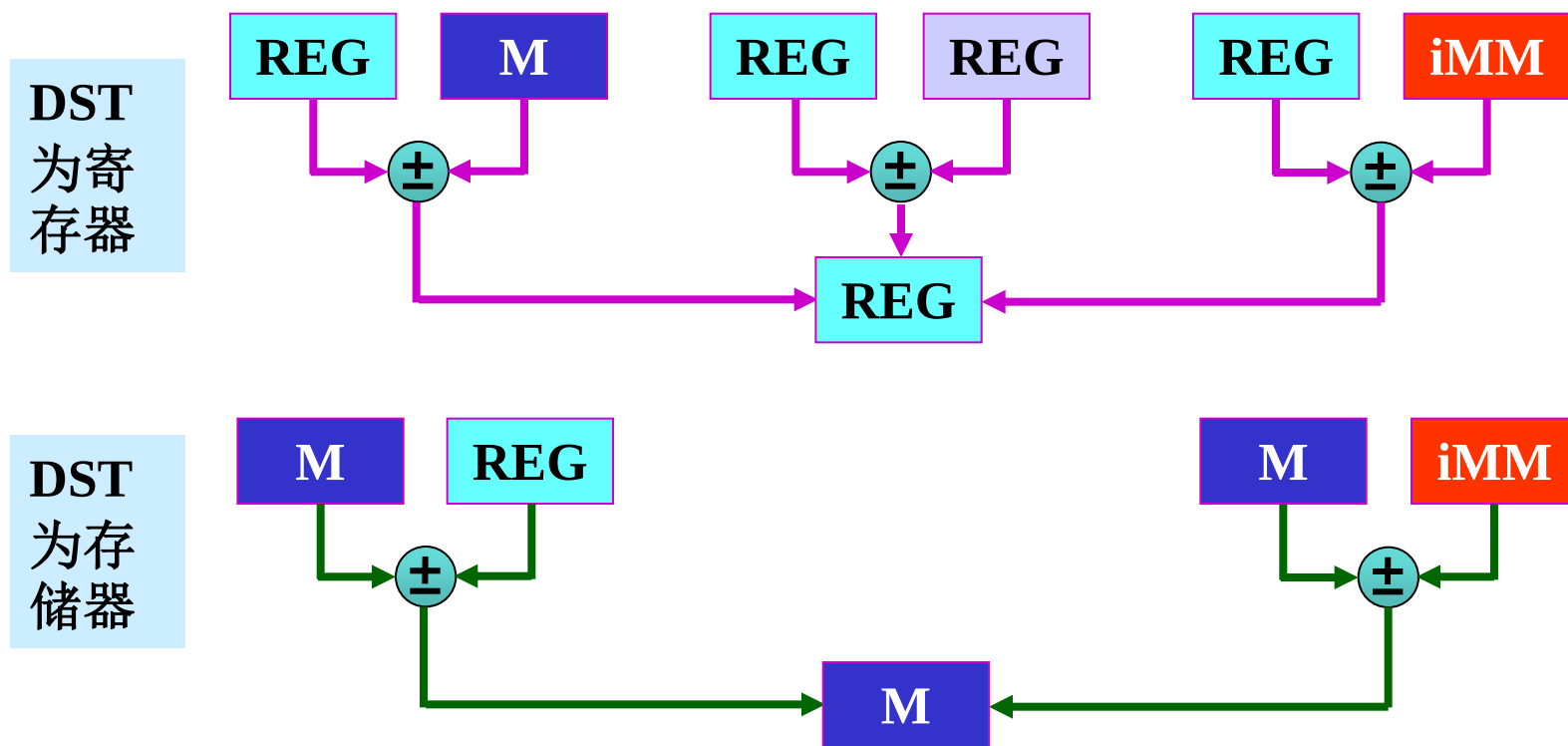
4.5.1 数据传送指令

- **9. IN/OUT 输入/输出**（不需要段地址）
 - (1) **IN 输入指令** (input)
 - 1) **直接寻址**（端口地址 $n \leq 255$ ）
IN AL, n ; 字节, (AL) $\leftarrow$ (n)
 - 2) **间接寻址**（端口地址在DX中，全部端口地址可用）
IN AL, DX ; 字节, (AL) $\leftarrow$ ((DX))
 - (2) **OUT 输出指令** (output)
 - 1) **直接寻址**（端口地址 $n \leq 255$ ）
OUT n, AL ; 字节, (n) $\leftarrow$ (AL)
 - 2) **间接寻址**（全部端口地址可用）
OUT DX, AL ; 字节, ((DX)) $\leftarrow$ (AL)



4.5.2 算术运算指令

- 1. 加法、减法指令
- 1) **ADD** 加法 (add)
- 格式: **ADD DST, SRC** ; $(DST) \leftarrow (SRC) + (DST)$
- 类型: 影响CF、PF、AF、ZF、SF、OF。可进行8、16、32位操作。





4.5.2 算术运算指令

- **2) ADC 带进位加法** (add with carry)
 - 格式: `ADC DST, SRC` ; $(DST) \leftarrow (SRC) + (DST) + CF$
- **3) SUB 减法** (subtract)
 - 格式: `SUB DST, SRC` ; $(DST) \leftarrow (DST) - (SRC)$
- **4) SBB 带借位减法** (subtract with borrow)
 - 格式: `SBB DST, SRC` ; $(DST) \leftarrow (DST) - (SRC) - CF$



4.5.2 算术运算指令

- 2. 加 1、减 1 指令
- 1) INC 加1 (increment)
- 格式: INC OPR ; $(\text{OPR}) \leftarrow (\text{OPR}) + 1$
- 2) DEC 减1 (decrement)
- 格式: DEC OPR ; $(\text{OPR}) \leftarrow (\text{OPR}) - 1$
- 2条指令共性说明:
 - 1) 目的操作数: 通用寄存器、存储器单元, 可以是8位、16位或32位。
 - 2) 执行INC、DEC指令后, 影响AF、OF、PF、SF、ZF, 但对CF没有影响。



4.5.2 算术运算指令

- **3. CMP 比较 (compare)**
- **格式:** `CMP OPR1, OPR2 ; (OPR1) - (OPR2)`
- **4. MUL 无符号数乘法 (unsigned multiple)**
- **格式:** `MUL SRC`
- **功能:**
 - 字节乘 $(AX) \leftarrow (AL) \times (SRC)$
 - 字乘 $(DX:AX) \leftarrow (AX) \times (SRC)$
- **5. DIV 无符号数除法 (unsigned divide)**
- **格式:** `DIV SRC`
- **功能:** 字节除 $(AL) \leftarrow (AX)/(SRC)$ 的商, $(AH) \leftarrow (AX)/(SRC)$ 的余数
字除 $(AX) \leftarrow (DX:AX)/(SRC)$ 的商, $(DX) \leftarrow (DX:AX)/(SRC)$ 的余数



4.5.3 BCD 码调整指令

压缩BCD码

7	4	3	0
BCD		BCD	

非压缩BCD码

7	4	3	0
0000		BCD	

- **1. DAA 压缩BCD码加法调整** (decimal adjust for addition)
- 格式: DAA
- 功能:
 - 1) 如果AL的低4位大于9或AF=1, 则(AL)+6 → (AL) 和 1→AF。
 - 2) 如果AL的高4位大于9或CF=1, 则(AL)+60H → (AL) 和 1→CF。
- 说明:
 - 1) 只对AL调整。
 - 2) OF无定义, 其他5个状态受影响。
- 例
- MOV AL, 54H ; 54H代表十进制数54
MOV BL, 37H ; 37H代表十进制数37
ADD AL, BL ; AL中的和为十六进制数8BH
DAA ; AL=91H, AF=1, CF=0 , 十进制加法和为91。



4.5.3 BCD 码调整指令

- **2. DAS 压缩BCD码减法调整** (decimal adjust for subtraction)
 - 格式: DAS
 - 功能: 调整压缩BCD码减法结果
- **3. AAA 非压缩BCD码加法调整** (ASCII adjust for addition)
 - 格式: AAA
 - 功能: 调整非压缩BCD码加法结果
- **4. AAS 非压缩BCD码减法调整** (ASCII adjust for subtraction)
 - 格式: AAS
 - 功能: 调整非压缩BCD码减法结果



4.5.4 逻辑运算指令

- 1. 与、或、异或、比较逻辑运算指令
- 1) AND 按位逻辑与运算
- 格式: **AND DST, SRC ; (DST) $\leftarrow$ (DST) $\wedge$ (SRC)**
- 2) OR 按位逻辑或运算
- 格式: **OR DST, SRC ; (DST) $\leftarrow$ (DST) $\vee$ (SRC)**
- 3) XOR 按位逻辑异或运算 (**exclusive or**)
- 格式: **XOR DST, SRC ; (DST) $\leftarrow$ (DST) $\oplus$ (SRC)**
- 4) TEST 按位逻辑比较运算
- 格式: **TEST OPR1, OPR2 ; (OPR1) $\wedge$ (OPR2)**
- 4条指令类型: 同ADD。
- 4条指令说明: **CF、OF置0**, 影响SF、ZF、PF, AF无定义。



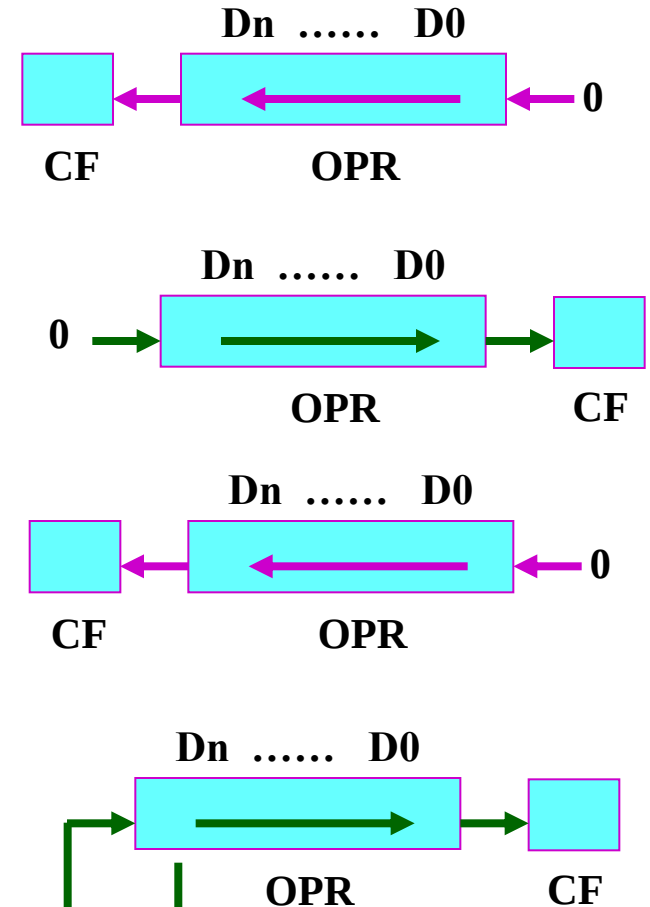
4.5.4 逻辑运算指令

- 2. NOT 按位取反指令
- 格式: NOT OPR ; $(\text{OPR}) \leftarrow \overline{\text{OPR}}$
- 类型: 通用寄存器、存储器单元, 操作数可以是8位、16位、32位。
- 说明: 操作数OPR每一位取反, 不影响标志位。



4.5.5 移位循环指令

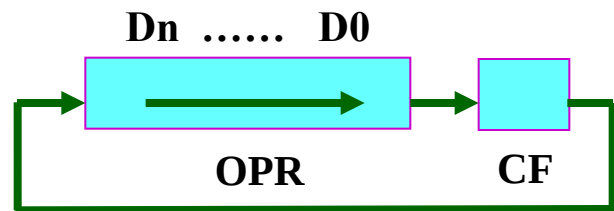
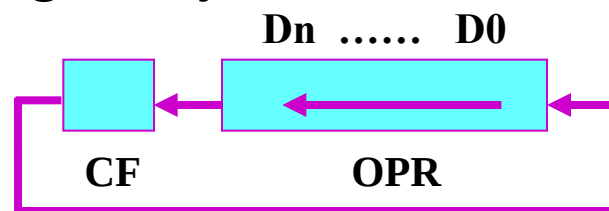
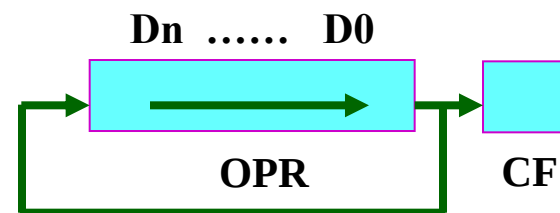
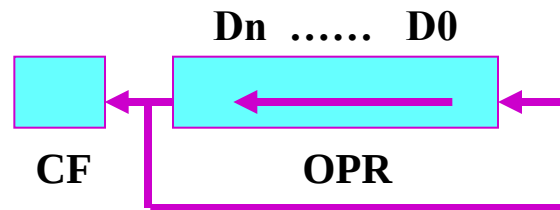
- 1. 移位指令
- 1) SHL 逻辑左移 (shift logical left)
- 格式: SHL OPR, CNT
- 2) SHR 逻辑右移 (shift logical right)
- 格式: SHR OPR, CNT
- 3) SAL 算术左移 (shift arithmetic left)
- 格式: SAL OPR, CNT
- 4) SAR 算术右移 (shift arithmetic right)
- 格式: SAR OPR, CNT
- 移位指令说明:
 - 1) OPR: 通用寄存器、存储器单元。8位、16位、32位。
 - 2) CNT: 8位立即数 (1-31, 8086中只能为1)、CL。
 - 3) CF依操作设置。OF当CNT=1时有效。





4.5.5 移位循环指令

- 2. 循环指令
- 1) **ROL** 循环左移 (rotate left)
- 格式: **ROL OPR, CNT**
- 2) **ROR** 循环右移 (rotate right)
- 格式: **ROR OPR, CNT**
- 3) **RCL** 带进位循环左移 (rotate left through carry)
- 格式: **RCL OPR, CNT**
- 4) **RCR** 带进位循环右移 (rotate right through carry)
- 格式: **RCR OPR, CNT**





4.5.6 控制转移指令

- 1. 无条件段内相对短转移 (jump)
 - 格式: **JMP SHORT OPR**
 - 功能: $(IP) \leftarrow (IP) + 8\text{位位移量}$
 - 说明: **SHORT**是短转移属性描述符, 8位位移量是一个带符号数, 范围是-128到+127字节。
-
- 2. 无条件段内相对近转移
 - 格式: **JMP NEAR PTR OPR**
 - 功能: $(IP) \leftarrow (IP) + 16\text{位位移量}$
 - 说明: **NEAR**是近转移属性描述符, 16位位移量是一个带符号数。



4.5.6 控制转移指令

- 3. 无条件段内间接转移
 - 格式: **JMP OPR** ; OPR可以是寄存器、存储器
 - 功能: $(IP) \leftarrow (EA)$ 或 $(EIP) \leftarrow (EA)$
- 4. 无条件段间直接转移
 - 格式: **JMP FAR PTR OPR** ; OPR为指令的标号（即指令地址）
 - 功能: $(IP/EIP) \leftarrow OPR$ 的段内偏移地址
 $(CS) \leftarrow OPR$ 所在段的段地址
- 5. 无条件段间间接转移
 - 格式: **JMP DWORD PTR OPR** ; OPR为存储器单元
 - 功能: $(IP/EIP) \leftarrow [EA]$
 $(CS) \leftarrow [EA+2]$



4.5.6 控制转移指令

- 6. 条件相对转移 (8位偏移量)

- 1) 单条件相对转移

① JZ / JE OPR ; ZF=1 转移(结果为零或相等转移)

② JNZ / JNE OPR ; ZF=0 转移(结果不为零或不相等转移)

③ JS OPR ; SF=1 转移(结果为负转移)

④ JNS OPR ; SF=0 转移(结果为正转移)

⑤ JO OPR ; OF=1 转移(结果溢出转移)

⑥ JNO OPR ; OF=0 转移(结果不溢出转移)

⑦ JP / JPE OPR ; PF=1 转移(结果为偶转移)

⑧ JNP / JPO OPR ; PF=0 转移(结果为奇转移)

⑨ JC OPR ; CF=1 转移(有借位或有进位转移)

⑩ JNC OPR ; CF=0 转移(无借位或无进位转移)

- J:jump Z:zero E:equal N:not S:sign P:parity C:carry
O:overflow PE:parity even PO:parity odd



4.5.6 控制转移指令

- 2) 无符号数比较条件相对转移(A-B) (A:above高于 B:below低于)
 - ① JB / JNAE / JC OPR ; CF=1 (A<B转移)
 - ② JAE / JNB / JNC OPR ; CF=0 (A≥B转移)
 - ③ JBE / JNA OPR ; (CF ∨ ZF)=1 (A≤B转移)
 - ④ JA / JNBE OPR ; (CF ∨ ZF)=0 (A>B转移)
- 3) 有符号数比较条件相对转移(A-B) (G:greater大于 L:less小于)
 - ① JL / JNGE OPR ; (SF ⊕ OF)=1 (A<B转移)
 - ② JGE / JNL OPR ; (SF ⊕ OF)=0 (A≥B转移)
 - ③ JLE / JNG OPR ; ((SF ⊕ OF) ∨ ZF)=1 (A≤B转移)
 - ④ JG / JNLE OPR ; ((SF ⊕ OF) ∨ ZF)=0 (A>B转移)



4.5.6 控制转移指令

- **7. LOOP 循环控制相对转移**
- **格式:** LOOP OPR
- **功能:** $(CX/ECX) \leftarrow (CX/ECX) - 1; (CX/ECX) \neq 0$ 转移
- **说明:** 上述循环控制相对转移指令中的转移地址为 (IP/EIP)
 $\leftarrow (IP/EIP) + 8$ 位带符号数; 8 位位移量是由目标地址 OPR 确定的。



4.5.6 控制转移指令

- **8. 子程序调用与返回**
- 子程序：具有独立功能的程序模块。
- **1) 段内相对调用**
- **格式：**CALL DST ； DST为直接入口地址，指令中给出
CALL NEAR PTR DST
- **功能：**
 - 操作数16位：
$$SP \leftarrow (SP) - 2, [SP] \leftarrow (IP)$$
$$IP \leftarrow (IP) + 16\text{位位移量}$$
 - 操作数32位：
$$ESP \leftarrow (ESP) - 4, [ESP] \leftarrow (EIP)$$
$$EIP \leftarrow (EIP) + 32\text{位位移量}$$
- **说明：**16位/32位位移量是一个有符号数。



4.5.6 控制转移指令

- 2) 段内间接调用
- 格式: **CALL DST** ; DST为R、M
- 功能:

操作数16位:

$SP \leftarrow (SP) - 2, [SP] \leftarrow (IP)$

$(IP) \leftarrow (EA)$ 、

操作数32位:

$ESP \leftarrow (ESP) - 4, [ESP] \leftarrow (EIP)$

$(EIP) \leftarrow (EA)$

- 类型: 寄存器、存储器单元。



4.5.6 控制转移指令

- 3) 段间直接调用
- 格式: **CALL DST** ; DST为直接入口地址, 指令中给出
- 功能:

操作数16位:

$SP \leftarrow (SP) - 2, [SP] \leftarrow (CS)$

$SP \leftarrow (SP) - 2, [SP] \leftarrow (IP)$

$IP \leftarrow \text{DST的偏移地址}$

$CS \leftarrow \text{DST所在段的段地址}$

操作数32位:

$ESP \leftarrow (ESP) - 2, [ESP] \leftarrow (CS)$

$ESP \leftarrow (ESP) - 4, [ESP] \leftarrow (EIP)$

$EIP \leftarrow \text{DST的偏移地址}$

$CS \leftarrow \text{DST所在段的段地址}$

- 说明: 该指令先将CS、IP或EIP压栈保护返回地址。然后转移到由DST (DST为汇编语言中的过程名) 指定的转移地址。



4.5.6 控制转移指令

- 4) 段间间接调用
- 格式: **CALL DST**; DST为存储器类型操作数
- 功能:

操作数16位:

$SP \leftarrow (SP) - 2, [SP] \leftarrow (CS)$

$SP \leftarrow (SP) - 2, [SP] \leftarrow (IP)$

$IP \leftarrow (EA)$

$CS \leftarrow (EA + 2)$

操作数32位:

$ESP \leftarrow (ESP) - 2, [ESP] \leftarrow (CS)$

$ESP \leftarrow (ESP) - 4, [ESP] \leftarrow (EIP)$

$EIP \leftarrow (EA)$

$CS \leftarrow (EA + 4)$

- 说明: 先保护返回地址, 然后转移到由DST指定的转移地址。EA由DST确定的任何内存寻址方式。



4.5.6 控制转移指令

- 5) 段内返回

- 格式: **RET**

- 功能:

操作数16位: $IP \leftarrow \text{栈弹出2字节}, SP \leftarrow (SP) + 2$

操作数32位: $EIP \leftarrow \text{栈弹出4字节}, ESP \leftarrow (ESP) + 4$

- 6) 段间返回

- 格式: **RET**

- 功能:

操作数16位: $IP \leftarrow \text{栈弹出2字节}, SP \leftarrow (SP) + 2$

$CS \leftarrow \text{栈弹出2字节}, SP \leftarrow (SP) + 2$

操作数32位: $EIP \leftarrow \text{栈弹出4字节}, ESP \leftarrow (ESP) + 4$

$CS \leftarrow \text{栈弹出2字节}, ESP \leftarrow (ESP) + 2$



4.5.6 控制转移指令

- 9. 中断指令
- 中断指令作用：调用中断服务程序。中断向量地址=中断类型码 $\times 4$ 。
- 1) INT 中断指令
- 格式：INT n ； 不影响标志位
- 功能：
 - ① $SP \leftarrow (SP) - 2$
 - ② PUSH (FR) ； 标志寄存器FR进栈 ①
 - ③ $SP \leftarrow (SP) - 2$
 - ④ PUSH (CS) ； 断点段地址CS进栈 ②
 - ⑤ $SP \leftarrow (SP) - 2$
 - ⑥ PUSH (IP) ； 断点地址指针IP进栈 ③
 - ⑦ $TF \leftarrow 0$ ； 禁止单步
 - ⑧ $IF \leftarrow 0$ ； 禁止中断 ④
 - ⑨ $IP \leftarrow [n \times 4]$ ； 转向中断服务程序 ⑤
 - ⑩ $CS \leftarrow [n \times 4 + 2]$



4.5.6 控制转移指令

- 2) **IRET**中断返回
- 格式: **IRET** ; 标志位随标志寄存器出栈操作而变
- 功能:
 - ① $IP \leftarrow \text{栈弹出2字节}$; 断点偏移量出栈
 - ② $SP \leftarrow (SP) + 2$
 - ③ $CS \leftarrow \text{栈弹出2字节}$; 断点段地址出栈
 - ④ $SP \leftarrow (SP) + 2$
 - ⑤ $FR \leftarrow \text{栈弹出2字节}$; 标志寄存器出栈
 - ⑥ $SP \leftarrow (SP) + 2$



4.5.6 控制转移指令

- **3) INT 21H 系统功能调用**
- **系统功能调用：**DOS为系统程序员及用户提供的一组中断服务程序。
- **DOS规定用中断指令INT 21H作为进入各功能调用中断服务程序的总入口，再为每个功能调用规定一个功能号，以便进入相应各个中断服务程序的入口。**
- **程序员使用系统功能调用的过程：**
 - 1) **AH ← 功能调用编号**
 - 2) **设置入口参数**
 - 3) **CPU执行 INT 21H**
 - 4) **给出出口参数**
- **DOS 提供的主要功能调用见表 4.5.1 。**



4.5.6 控制转移指令

- (1) **功能号: 1** ; 等待键盘输入, 并回送显示器
MOV AH, 1 ; (AH) ← 功能号01H
INT 21H ; 调用21H号软中断
- **说明:** 出口参数 (AL) = 键入字符的ASCII码。
- (2) **功能号: 2** ; 输出字符送显示器
MOV DL, 41H ; 入口参数: (DL) ← 字符A的ASCII码
MOV AH, 2 ; (AH) ← 功能号02H
INT 21H ; 在屏幕上显示输出字符A
- **说明:** 无出口参数。
- (3) **功能号: 4CH** ; 终止程序, 返回
MOV AH, 4CH ; (AH) ← 功能号4CH
INT 21H ; 调用21H号软中断



4.5.8 处理器控制指令

- 标志处理指令只设置或清除本指令的标志位，而不影响其他标志位。
- **CLC** ; 进位位置0, $CF \leftarrow 0$ (clear carry)
- **STC** ; 进位位置1, $CF \leftarrow 1$ (set carry)
- **CLD** ; 方向标志位置0, $DF \leftarrow 0$ (clear direction)
- **STD** ; 方向标志位置1, $DF \leftarrow 1$ (set direction)
- **CLI** ; 中断标志置0, $IF \leftarrow 0$, 禁止中断 (clear interrupt)
- **STI** ; 中断标志置1, $IF \leftarrow 1$, 允许中断 (set interrupt)
- **NOP** ; 空操作, 不执行任何操作。
- **HLT** ; 暂停, 暂停指令停止软件的执行。有三种方式退出暂停: 中断、硬件复位、DMA操作。



4.6 伪指令

- **伪指令功能：**指示汇编程序完成规定的操作。如选择处理器、定义数据、分配存储区等。
- **1. END 源程序结束伪指令**
- **格式：**END [标号]
- **功能：**表示源程序结束，**不可缺**，源程序最后一条语句。
- **说明：**
 - 1) 标号指示程序开始执行的起始地址。
 - 2) 主程序缺省值为代码段的第一条指令的地址。
 - 3) 多个模块链接，主程序用标号，其他程序不用。



4.6 伪指令

- **2. SEGMENT 和 ENDS**（段定义伪指令）
- **段定义：**确定代码组织与数据存储的方式。
- **格式：**段名 SEGMENT [定位类型] [组合类型] [字长类型] ['类别']
:
段名 ENDS
- **功能：**定义段名、段属性，并表示段的开始位置、结束位置。
- **说明：**
 - 1) **SEGMENT和ENDS必须成对出现，而且伪指令前面的段名也要相同**
 - 2) **段名是段的标识符，指明段的基址，由程序员指定。**
 - 3) 一般情况下，选项可以不用，使用默认值。但若需链接程序，就必须使用这些说明。



4.6 伪指令

- 2. SEGMENT 和 ENDS （段定义伪指令）

- 格式：

段名 SEGMENT [定位类型] [组合类型] [字长类型] [‘类别’]

⋮

段名 ENDS

- 定位类型： 指定段起始地址边界。
- 组合类型： 表示本段与其他模块段之间，具有相同段名的各段的组合关系。
- 字长类型： 386以后，说明使用16位寻址还是32位寻址。
- 类别： 引号括起来的字符串。连接时，类别相同的段（它们可能不同名）放在连续的存储空间中，但它们仍然是不同的段。



4.6 伪指令

- 3. ASSUME（段分配伪指令）

- 格式：

ASSUME 段寄存器名：段名，段寄存器名：段名，

- 功能：说明某个段使用哪一个段寄存器。

- 说明：

- 1) 程序段必须用CS，堆栈段必须用SS。

- 2) 该语句一般放在代码段最前面。

- 3) **说明性语句**，除CS外（初始化赋值），各段寄存器在程序中赋值。



4.6 伪指令

- **地址计数器\$**：指示当前语句使用段内存储单元的偏移值，每段开始时为0，每分配一个单元加1。
- **4. ORG 地址计数器设置（起始地址定义）**
- **格式**：ORG 数值表达式
- **功能**：定义指令或数据的起始地址， $\$ \leftarrow$ 表达式的值。
- **说明**：数值表达式取值范围在0~65535之间。
- **5. EVEN 偶数地址定义（使地址计数器成为偶数）**
- **格式**：EVEN
- **功能**：使下一个变量或指令从偶地址开始。



4.6 伪指令

- 6. 数据定义伪指令

- 格式:

[变量名] 操作符 操作数 [; 注释]

- 功能: 为操作数分配存储单元, 用变量与存储单元相联系。

- 变量名和注释是可有可无的。

- 操作符: 定义操作数类型。

DB: 一个操作数占有1个字节单元 (8位), 定义的变量为字节变量。

DW: 一个操作数占有1个字单元 (16位), 定义的变量为字变量。

DD: 一个操作数占有1个双字单元 (32位), 定义的变量为双字变量。

操作数: 常数、表达式、字符串、? 等。



例 常数与表达式

- 例 操作数为常数与表达式

ORG 200H

DATA1 DB 12H, 2+6, 34H

EVEN

DATA2 DW 789AH

ALIGN 4

DATA3 DD 12345678H

DATA4 DW \$, 6699H

汇编结果		
变量名	偏移量	存储单元内容
DATA1→	200H	12H
	201H	08H
	202H	34H
	203H	(即保留原值)
DATA2→	204H	9AH
	205H	78H
	206H	(即保留原值)
	207H	(即保留原值)
DATA3→	208H	78H
	209H	56H
	20AH	34H
	20BH	12H
DATA4→	20CH	0CH
	20DH	02H
	20EH	99H
	20FH	66H



例 “？”

- 例 操作数是 “？” 定义形式。
- 此时只分配存储空间，
但不定义初值。

ORG 400H

DATA1 DB 1, 2, ?, 4

DATA2 DW 5, ?, 6

汇编结果

变量名	偏移量	存储单元内容
DATA1→	400H	01H
	401H	02H
	402H	(即保留原值)
	403H	04H
DATA2→	404H	05H
	405H	00H
	406H	(即保留原值)
	407H	(即保留原值)
	408H	06H
	409H	00H
DATA3→	40AH	(即保留原值)
	40BH	(即保留原值)
	40CH	(即保留原值)
	40DH	(即保留原值)
	40EH	(即保留原值)
	40FH	(即保留原值)
DATA4→	410H	08H



例 DUP

- 例 操作数用复制操作符DUP定义形式
- 此时表示操作数重复若干次。

ORG 300H

DATA1 DB 2 DUP (12H, 34H, 56H)

汇编结果		
变量名	偏移量	存储单元内容
DATA1→	300H	12H
	301H	34H
	302H	56H
	303H	12H
	304H	34H
	305H	56H



4.6 伪指令

- **7. PROC 和 ENDP**（过程定义伪指令）

- 格式：

过程名 **PROC** [属性]

⋮ （过程体）

过程名 **ENDP**

- 功能：用于定义子程序结构，**定义一段程序的入口（过程名）及属性。**
- 说明：过程名是该过程（子程序）的入口，是CALL的操作数。
- 属性：**FAR、NEAR**（默认）



4.6 伪指令

- **8. EQU**（赋值伪指令）
- **格式：**符号常数名 EQU 表达式
- **功能：**将表达式的值赋给符号常数。
- **说明：**表达式可以是有效的操作数格式，也可以是任何可求出数值常数的表达式，还可以是任何有效的符号（如操作符、寄存器名、变量名等）。
- **注意：**只能定义一次。
- **例：**

DATA1 EQU 88

AAA1 EQU CX

DATA2 EQU DATA1+12



4.6 伪指令

• 9. 返回值运算符

- **返回值运算符**：返回变量或标号的段地址（SEG）、返回变量或标号的偏移地址（OFFSET）、返回变量或标号的类型值（TYPE）、返回变量的单元数（LENGTH）、返回变量的字节数（SIZE）。
- **格式及功能**：
 - 操作数 SEG 变量/标号 ; 段地址值赋给操作数。
 - 操作数 OFFSET 变量/标号 ; 偏移量值赋给操作数。
 - 操作数 TYPE 变量/标号 ; 代表变量/标号类型的值赋给操作数。
 - 操作数 TYPE 变量 ; 返回变量数据类型的字节数：DB为1、DW为2、DD为4。
 - 操作数 TYPE 标号 ; 标号为NEAR类型时返回-1，为FAR类型时返回-2。
 - 操作数 LENGTH 变量 ; 第一个数占用的单元数赋给操作数。
 - 操作数 SIZE 变量 ; 第一个数占用的字节数赋给操作数。



4.6 伪指令

- **10. PTR**（临时改变类型属性运算符）
- **格式：**类型 **PTR** 变量/标号 ； 临时改变类型属性
- **说明：**变量：字节**BYTE**、字**WORD**、双字**DWORD**，等；
标号：近类型**NEAR**、远类型**FAR**。

- **例**

```
DATA1    DW    1234H, 5678H
DATA2    DB    99H, 88H, 77H, 66H
MOV      AX, WORD PTR DATA2 ; 8899H→(AX)
MOV      BL, BYTE PTR DATA1  ; 34H→(BL)
MOV      DX, DATA1+2          ; 5678H→(DX)
MOV      BYTE PTR [BX], 8     ; 存入字节单元
MOV      WORD PTR [BX], 8     ; 存入字单元
```



4.7 汇编语言源程序的结构

80x86汇编语言源程序结构特点：

- (1) 一个程序由若干逻辑段组成，各逻辑段由伪指令语句定义和说明。
- (2) 整个源程序以END伪指令结束。
- (3) 每个逻辑段由语句序列组成，语句可以是：

指令语句：CPU指令，可执行语句。汇编时译成目标码。

伪指令语句：CPU不执行，提供汇编信息，不产生目标代码。

宏指令语句：一个指令序列，汇编时产生对应的目标代码序列。

注释语句：以分号“；”开始，说明性语句，汇编程序不予处理。

空行语句：保持程序书写清晰，仅包含回车换行符。

- 每个源程序应当有返回操作系统的指令语句，使程序执行完后能自动返回系统。



4.8 汇编语言程序设计

汇编语言程序设计的基本步骤:

- (1) 描述问题
- (2) 确定算法。
- (3) 绘制流程图。
- (4) 分配存储空间和工作单元。
- (5) 编写程序。
- (6) 上机调试。

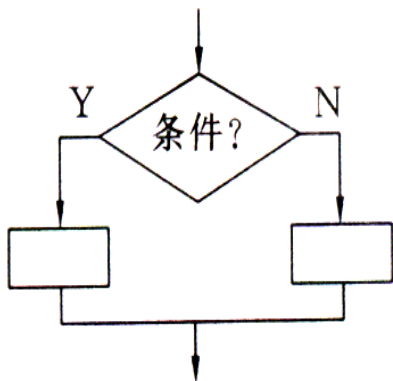
程序基本结构形式:

顺序、分支、循环、子程序。

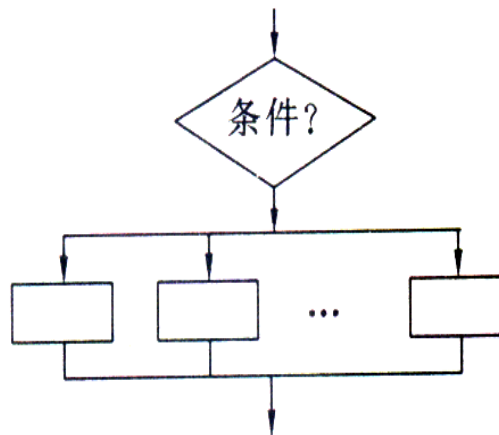


4.8.1 分支程序设计

- **分支程序**：根据不同的情况或条件执行不同的功能，具有判断和转移功能，在程序中利用条件转移指令对运算结果的状态标志进行判断实现转移。
- **特点**：按条件判断、转移、分支。
- **分支程序有2种结构形式**：二路分支结构、多路分支结构。
- 两种结构的**共同特点**：在某一特定条件下，只执行多个分支中的一个分支。



二路分支结构



多路分支结构



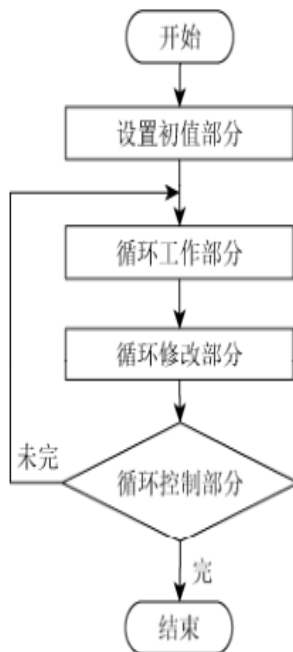
4.8.2 循环程序设计

- 循环程序由五部分组成：

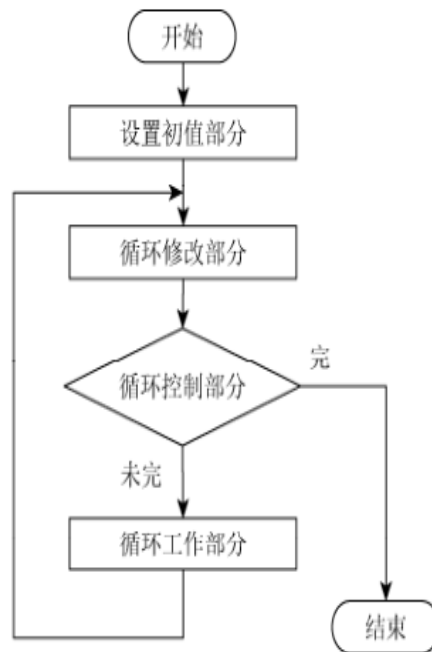
- (1) 初始化部分：设置初值。
- (2) 循环工作部分：核心。
- (3) 循环修改部分：修改循环工作变量、指针。
- (4) 循环控制部分：修改控制变量，确定程序走向。
- (5) 循环结束部分：结果处理。

- 常用结构2种：

(1) 先执行，后判断。至少执行一次循环体。



(a) 先执行后判断结构



(b) 先判断后执行结构

(2) 先判断，后执行。允许零次循环。

图 4.8.1 循环程序结构



例 4.8.4 多重循环程序设计

- 循环程序设计举例-例4.8.4: 无符号数从大到小排序。从首地址NUMBER1开始存放N个无符号字节数据, 用冒泡法编制将数据按由大到小重新排序的源程序。
 - 冒泡排序法: N个数据, 相邻两个单元大小判断, 位置交换, N-1次后, 排出最大(或最小)数据。经过N-1次的上述循环, 便可实现数据排序。
 - 关键步骤: 比较, 判断, 交换。
 - 判断指令:

顺序	有符号	无符号
由大到小	大于 JGE	高于 JAE JNC
由小到大	小于 JLE	低于 JBE

小地址	最大 //////// ////////
大地址	最小

由大到小排序

小地址	最小 //////// ////////
大地址	最大

由小到大排序



例4.8.4 - (1) N-1遍，每遍比较N-1次

DATA SEGMENT

NUMBER1 DB 100, 3, 90, 80, 99, 77, 44, 66, 50 ; 9个数据

N EQU \$- NUMBER1-1 ; 数据个数减1 (8个), \$是汇编指针

DATA ENDS

CODE SEGMENT

ASSUME CS:CODE, DS:DATA

START: MOV AX, DATA }
MOV DS, AX }

MOV DX, N ; 外循环计数初值，即遍数

A1: LEA BX, NUMBER1 ; 数据首址

MOV CX, N ; 内循环计数初值，即比较次数

A2: { MOV AL, [BX]

{ CMP AL, [BX+1] ; 相邻两个单元比较

JNC A3 ; 前大后小转 (小地址数大, 大地址数小)

{ XCHG AL, [BX+1] ; 前小后大, 交换

{ MOV [BX], AL

A3: INC BX ; 形成下一个比较地址

LOOP A2 ; 内循环计数判转，即比较次数

DEC DX ; 外循环计数，即计遍数

JNZ A1 ; 外循环判转

MOV AH, 4CH }

INT 21H }

CODE ENDS

END START



例4.8.4 - (2) N-1遍，每遍比较次数少1

```
DATA    SEGMENT
NUMBER1 DB    100, 3, 90, 80, 99, 77, 44, 66, 50  ; 9个数据
N        EQU    $- NUMBER1-1      ; 数据个数减1 (8个), $是汇编指针
DATA    ENDS
CODE    SEGMENT
        ASSUME CS:CODE, DS:DATA
START:  MOV     AX, DATA
        MOV     DS, AX
        MOV     DX, N              ; 外循环计数初值, 即遍数
A1:     LEA     BX, NUMBER1        ; 数据首址
        MOV     CX, DX            ; 内循环计数初值, DX内容每循环一次减1
A2:     MOV     AL, [BX]
        CMP     AL, [BX+1]        ; 比较
        JNC     A3                ; 前大后小转
        XCHG    AL, [BX+1]        ; 前小后大交换
        MOV     [BX], AL
A3:     INC     BX                ; 形成下一个比较地址
        LOOP    A2                ; 内循环计数判转, 即比较次数
        DEC     DX                ; 外循环计数, 即计遍数
        JNZ     A1                ; 外循环判转
        MOV     AH, 4CH
        INT     21H
CODE    ENDS
        END     START
```



例4.8.4 - (3) 有交换标志, N-1遍, 每遍比较次数少1

CODE SEGMENT
 ASSUME CS:CODE, DS:DATA
START: MOV AX, DATA
 MOV DS, AX
 MOV DX, N ; 外循环计数初值, 即遍数
A1: LEA BX, NUMBER1 ; 数据首址
 SUB AH, AH ; 设置交换标志, 初值0
 MOV CX, DX ; 内循环计数初值, 即比较次数
A2: MOV AL, [BX]
 CMP AL, [BX+1] ; 比较
 JNC A3 ; 前大后小转
 XCHG AL, [BX+1] ; 前小后大交换
 MOV [BX], AL
 MOV AH, 1 ; 设置有关标志=1
A3: INC BX ; 形成下一个比较地址
 LOOP A2 ; 内循环计数判转, 即比较次数
 OR AH, AH ; 交换标志的反码送ZF标志位
 JZ A4 ; 无交换转
 DEC DX ; 外循环计数, 即计遍数
 JNZ A1 ; 外循环判转
A4: ;
 END START

设置

改变

判断



4.8.3 子程序设计

1. 子程序的定义

子程序组成：7部分，子程序定义（即子程序入口）、保护现场、取入口参数、子程序体、存输出参数、恢复现场、返回。

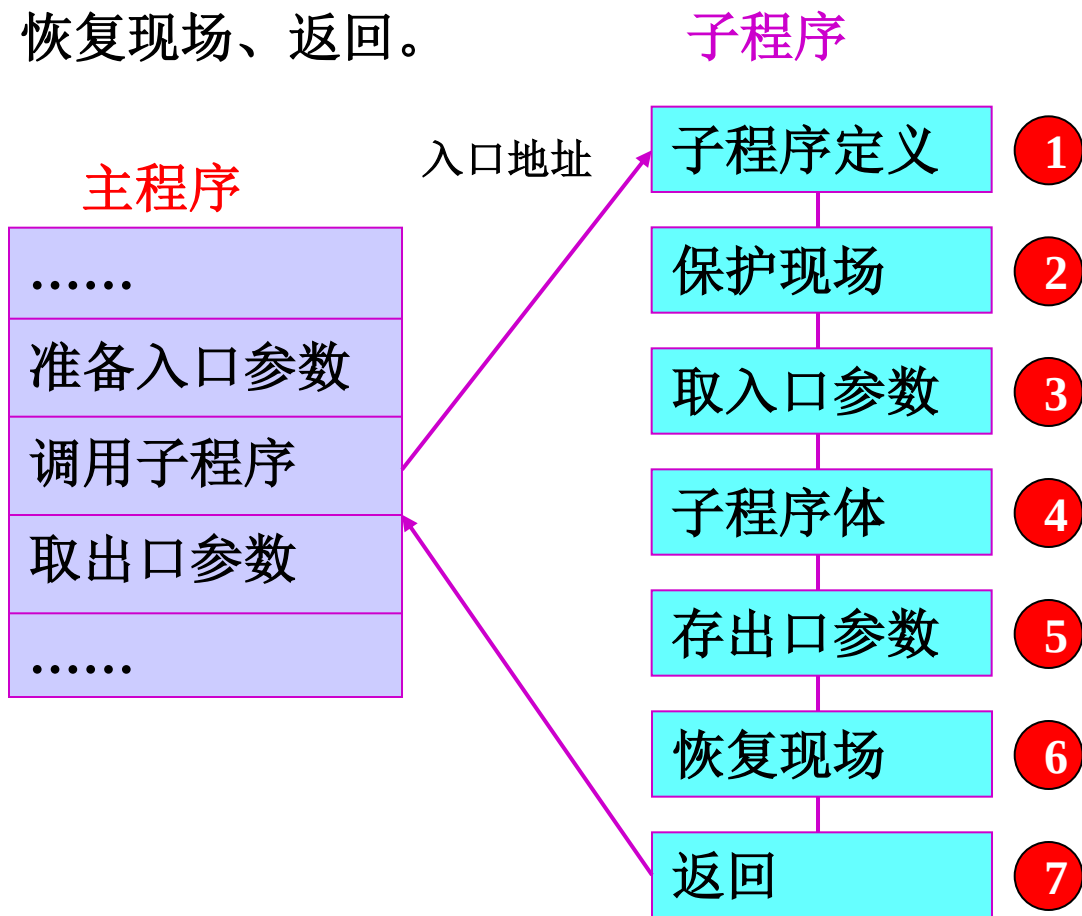
2. 现场的保存与恢复

保护现场：在子程序中，保护那些主程序要用、而子程序也要用的寄存器内容。

恢复现场：恢复现场保护的内容。

现场保护和现场恢复的方法：

- (1) 使用堆栈指令
- (2) 使用数据传送指令





结 束